

Chapter 10

Set Interface

A **Set** is a collection that **does not allow duplicate elements**.

Main features

- Does not store duplicate values.
- Can store multiple elements.
- Order depends on the implementation.
- Common implementations:
 - HashSet
 - LinkedHashSet
 - TreeSet

Example

```
import java.util.*;

public class SetExample {

    public static void main(String[] args) {

        Set<String> fruits = new HashSet<>();

        fruits.add("Apple");

        fruits.add("Mango");

        fruits.add("Apple"); // Duplicate

        System.out.println(fruits);

    }

}
```

Output:

[Mango, Apple]

The second "Apple" is not added because Set does not allow duplicates.

Implementation of Set

Set is an **interface**, so we cannot directly create its object.

We use classes such as:

```
Set<String> set = new HashSet<>();
```

Here:

- Set → interface
- HashSet → implementation class

Other implementations:

```
Set<String> set1 = new HashSet<>();
```

```
Set<String> set2 = new LinkedHashSet<>();
```

```
Set<String> set3 = new TreeSet<>();
```

HashSet

HashSet is an implementation of the Set interface.

Features

- Does not allow duplicates.
- Does not maintain insertion order.
- Allows one null value.
- Fast for adding, removing and searching elements.

Example

```
import java.util.*;
```

```
public class HashSetExample {
```

```
public static void main(String[] args) {  
  
    HashSet<String> names = new HashSet<>();  
  
    names.add("Arun");  
    names.add("Bala");  
    names.add("Kumar");  
    names.add("Arun");  
  
    System.out.println(names);  
}  
}
```

Output may be:

[Arun, Kumar, Bala]

The order is not guaranteed.

LinkedHashSet

LinkedHashSet is similar to HashSet, but it **maintains insertion order**.

Features

- Does not allow duplicates.
- Maintains insertion order.
- Allows one null value.
- Slightly slower than HashSet.

Example

```
import java.util.*;  
  
public class LinkedHashSetExample {  
  
    public static void main(String[] args) {
```

```
LinkedHashSet<String> names = new LinkedHashSet<>();

names.add("Arun");

names.add("Bala");

names.add("Kumar");

names.add("Arun");

System.out.println(names);

}

}
```

Output:

[Arun, Bala, Kumar]

The original insertion order is maintained.

TreeSet

TreeSet is a Set implementation that stores elements in **sorted order**.

Features

- Does not allow duplicates.
- Stores elements in ascending order by default.
- Does not allow null in normal natural-order usage.
- Useful when sorted data is required.

Example

```
import java.util.*;

public class TreeSetExample {

    public static void main(String[] args) {
```

```
TreeSet<Integer> numbers = new TreeSet<>();

numbers.add(50);

numbers.add(20);

numbers.add(40);

numbers.add(10);

numbers.add(20);

System.out.println(numbers);

}

}
```

Output:

[10, 20, 40, 50]

Notice that:

- Duplicate 20 is removed.
- Numbers are automatically sorted.

Map Interface

A **Map** stores data in the form of **key-value pairs**.

For example:

101 → Arun

102 → Bala

103 → Kumar

Here:

- 101 → key
- Arun → value

Features

- Keys must be unique.
- Values can be duplicated.
- Data is accessed using keys.
- Common implementations:
 - HashMap
 - LinkedHashMap
 - TreeMap

Example

```
import java.util.*;
```

```
public class MapExample {
```

```
    public static void main(String[] args) {
```

```
        Map<Integer, String> students = new HashMap<>();
```

```
        students.put(101, "Arun");
```

```
        students.put(102, "Bala");
```

```
        students.put(103, "Kumar");
```

```
        System.out.println(students);
```

```
        System.out.println("Student 102: " + students.get(102));
```

```
    }
```

```
}
```

Output:

```
{101=Arun, 102=Bala, 103=Kumar}
```

```
Student 102: Bala
```

Important Map methods

put()

get()

remove()

containsKey()

containsValue()

size()

Queue Interface

A **Queue** is used to store elements where processing generally follows **FIFO**.

FIFO = First In, First Out

Example:

Person 1 → Person 2 → Person 3

↓

Person 1 is served first

Common Queue methods

Method	Purpose
add()	Adds element
offer()	Adds element
remove()	Removes first element
poll()	Removes first element
peek()	Views first element

Example

```
import java.util.*;

public class QueueExample {

    public static void main(String[] args) {

        Queue<String> queue = new LinkedList<>();

        queue.add("A");
        queue.add("B");
        queue.add("C");

        System.out.println(queue);

        System.out.println("Removed: " + queue.poll());

        System.out.println("Queue: " + queue);
    }
}
```

Output:

[A, B, C]

Removed: A

Queue: [B, C]

Implementation of Queue

Queue is an interface, so we use implementation classes.

Common implementations include:

```
Queue<String> q = new LinkedList<>();
```

and

```
Queue<String> q = new PriorityQueue<>();
```

LinkedList as Queue

```
import java.util.*;
```

```
public class QueueImplementation {
```

```
    public static void main(String[] args) {
```

```
        Queue<Integer> numbers = new LinkedList<>();
```

```
        numbers.offer(10);
```

```
        numbers.offer(20);
```

```
        numbers.offer(30);
```

```
        System.out.println(numbers);
```

```
        System.out.println("First element: " + numbers.peek());
```

```
        System.out.println("Removed: " + numbers.poll());
```

```
        System.out.println(numbers);
```

```
    }
```

```
}
```

Output:

```
[10, 20, 30]
```

First element: 10

Removed: 10

[20, 30]

PriorityQueue

PriorityQueue is a Queue implementation where elements are processed according to their **priority**, rather than simply insertion order.

For numbers, the **smallest element has the highest priority by default**.

Example

```
import java.util.*;
```

```
public class PriorityQueueExample {  
    public static void main(String[] args) {
```

```
        PriorityQueue<Integer> queue = new PriorityQueue<>();
```

```
        queue.add(40);
```

```
        queue.add(10);
```

```
        queue.add(30);
```

```
        queue.add(20);
```

```
        System.out.println("Priority Queue: " + queue);
```

```
        System.out.println("Removed: " + queue.poll());
```

```
        System.out.println("Removed: " + queue.poll());
```

```
        System.out.println("Queue: " + queue);
```

```
}
```

```
}
```

Output:

Priority Queue: [10, 20, 30, 40]

Removed: 10

Removed: 20

Queue: [30, 40]

Quick Comparison

Topic	Main Purpose	Duplicate?	Order
Set	Store unique values	✗ No	Depends on implementation
HashSet	Unique values	✗ No	No guaranteed order
LinkedHashSet	Unique values	✗ No	Insertion order
TreeSet	Unique + sorted values	✗ No	Sorted order
Map	Key-value pairs	Keys ✗	Depends on implementation
Queue	Process elements	✓ Yes	Usually FIFO
LinkedList Queue	FIFO processing	✓ Yes	Insertion order
PriorityQueue	Priority-based processing	✓ Yes	Priority order